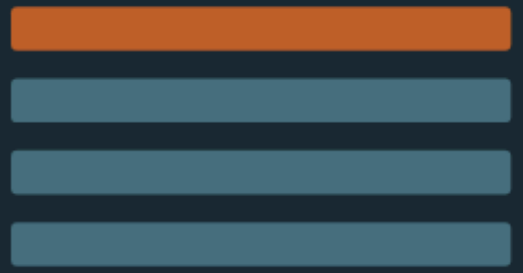
MAESTRÍA · ESPACIO ACADÉMICO

Gestión de Configuración y Mantenimiento de Software

Sesión 3 — Unidad 3 · Control de versiones y automatización (DevOps)

Cuando la velocidad y el control dejan de ser enemigos.



César Augusto Vega Fernández

Encuentro sincrónico · Git · Estrategias de ramas · Versionamiento semántico · CI/CD · Métricas de entrega

EL RETO DE LA UNIDAD 3 · ACTIVIDAD EN LA PLATAFORMA

De la estrategia en papel a un pipeline que funciona

«La estrategia de gestión de configuración está aprobada. Ahora queremos verla funcionando: cada cambio revisado y probado automáticamente, y cada versión identificada y entregable con un solo paso.»

Responsable del sistema que eligió cada grupo

Actividad 3 · Implementación de control de versiones y automatización

· 30 % · 24 horas · grupal, de 2 a 3

· Repositorio en Git · pipeline de CI/CD funcional

· Documento técnico de 5 a 7 páginas o README.md

· Evidencias: capturas, registros de ejecución, commits

Coherencia con la Unidad 2

· Las ramas, las versiones, las líneas base y el control de cambios del F-09 al F-11 son lo que ahora se implementa.

La actividad pide evidenciar el funcionamiento real del pipeline. Un archivo de configuración que nunca corrió no es un pipeline.

Los formatos de esta unidad son del F-15 al F-21. En el repositorio hay una plantilla con un pipeline que ya funciona, para copiar y adaptar.

CÓMO VAMOS A USAR LA SESIÓN

Primero qué significa, después ejemplos, después ustedes

PRIMERO

Conceptos

Git, estrategias de ramas, versionamiento semántico, integración continua, entrega continua y cómo se mide.

DESPUÉS

Ejemplos

Las ramas y entregas de Flask, y la plantilla del curso con su pipeline corriendo en GitHub.

Y EL RESTO DE LA SESIÓN

El hacer

El taller en salas: el repositorio del sistema con la rama principal protegida y el primer pipeline en verde.

El bloque que no se recorta es el tercero: salir de la sesión con un pipeline en verde ahorra la mitad de la actividad.

BLOQUE 1 · EL PROBLEMA

La tensión que DevOps intenta resolver

Desarrollo	Operaciones
Se le mide por las funcionalidades entregadas.	Se le mide por la ausencia de incidentes.
Quiere cambiar rápido.	Quiere que nada cambie.

La falsa sensación de control

Un pipeline configurado de afán no reduce la presión por entregar: a veces la aumenta, porque da una sensación de control que no siempre es real. Si las pruebas automáticas no cubren los casos que importan, el pipeline propaga errores tan rápido como correcciones.

Velocidad y control, en las condiciones correctas, no se oponen: se necesitan. Kim, Humble, Debois y Willis (2016).

BLOQUE 1 · GIT

Control distribuido e historia como documentación

Distribuido

Cada clon tiene el repositorio completo, con toda la historia.

Se puede trabajar, ramificar y consultar la historia sin un servidor central.

Trazabilidad

Cada commit registra qué cambió, quién y cuándo.

Con buenos mensajes, la historia es documentación técnica.

Con «fix» o «cambios varios», la historia existe pero no sirve.

Mensaje que no sirve	Mensaje que sirve
fix	fix(sesiones): respeta SESSION_COOKIE_DOMAIN al renovar la cookie (#212)
cambios varios	feat(api): agrega la ruta /version con el commit de la imagen

Chacon y Straub (2014), capítulos 3 y 5. El formato tipo: alcance: descripción es el de Conventional Commits 1.0.0.

BLOQUE 1 · ESTRATEGIAS DE RAMAS

El poder y el riesgo de las ramas

Estrategia	Cómo funciona	Conviene cuando	Riesgo
GitFlow	Ramas develop, de funcionalidad, de entrega y de corrección urgente, además de main	Entregas con fecha y varias versiones en mantenimiento	Muchas ramas largas: integraciones costosas
Flujo de GitHub	main siempre desplegable; una rama corta por cambio, que entra por PR	Entrega continua de una sola versión	Requiere pruebas automáticas confiables
Desarrollo en tronco	Todos integran a main varias veces al día; lo inacabado se oculta con interruptores	Equipos con mucha automatización	Exige disciplina y pruebas rápidas
Rama de mantenimiento	main para lo nuevo y una rama estable para corregir lo vigente	Hay usuarios en la versión anterior	Hay que llevar cada corrección a las dos ramas

El error más frecuente es elegir una estrategia porque es popular y no porque resuelva un problema concreto. La elección se justifica con la Unidad 2.

BLOQUE 1 · VERSIONAMIENTO SEMÁNTICO · HERRAMIENTA EN VIVO

Un contrato de comunicación con quien usa el sistema

MAYOR 2.0.0 → 3.0.0	Cambios que rompen la compatibilidad: retirar una función, cambiar una ruta, cambiar el formato de una respuesta.
MENOR 3.0.0 → 3.1.0	Funcionalidad nueva compatible. También marcar algo como obsoleto, antes de retirarlo.
PARCHE 3.1.0 → 3.1.1	Correcciones compatibles que no cambian la interfaz.

Para que el contrato valga

Hay que declarar qué es la interfaz pública.

Una versión publicada no se modifica: se publica otra.

Antes de 1.0.0 todo puede cambiar.

Preston-Werner, T. Semantic Versioning 2.0.0, https://semver.org. En el aula se usa el interactivo: doce cambios reales para decidir qué versión sigue.

BLOQUE 1 · INTEGRACIÓN CONTINUA

Si duele, hágalo con más frecuencia

1. **Integrar seguido**
   Los conflictos detectados el mismo día son baratos; los que se descubren tras semanas de trabajo divergente, no.
2. **Cada integración se verifica sola**
   Estilo, pruebas y construcción corren en cada PR, sin que nadie lo pida.
3. **Una integración rota se arregla primero**
   Mientras main esté en rojo, nadie integra otra cosa.
4. **Las pruebas tienen que poder fallar**
   Un pipeline que ejecuta validaciones triviales es un teatro de automatización.

Humble y Farley (2010)

«Si duele, hazlo con más frecuencia y trae el dolor hacia adelante.»

La integración continua es una práctica del equipo, no una herramienta.

BLOQUE 1 · ENTREGA CONTINUA

El pipeline de despliegue

Commit
Estilo y pruebas rápidas

›

Construcción
Un artefacto, una sola vez

›

Staging
El mismo artefacto, probado en un entorno

›

Aprobación
Quien tiene la autoridad (Unidad 2)

›

Producción
El mismo artefacto, entregado

Principio	Por qué
Construir una sola vez	Lo que se probó en staging es exactamente lo que llega a producción
Gestionar los artefactos	Paquetes e imágenes viven en un repositorio de artefactos, identificados por versión y hash
Poder volver atrás	Cada versión anterior sigue disponible y se sabe cómo desplegarla

Humble y Farley (2010). La gestión de artefactos es parte del saber de control de versiones del syllabus.

BLOQUE 1 · CÓMO SE MIDE

Cuatro métricas de entrega

Métrica	Qué mide	Cómo se aproxima con Git
Frecuencia de despliegue	Cada cuánto llega una versión a producción	Etiquetas de versión por mes o por año
Tiempo de entrega de los cambios	Cuánto tarda un commit en llegar a producción	Fecha de la etiqueta menos fecha del commit
Tasa de fallos de cambio	Qué proporción de despliegues causa un fallo	Proporción de versiones de corrección (aproximación gruesa)
Tiempo de restauración	Cuánto tarda en recuperarse el servicio	No se ve en Git: hace falta el registro de incidentes

Forsgren, Humble y Kim (2018)

Son las métricas DORA de Accelerate.

Las dos primeras miden velocidad y las dos últimas estabilidad.

Los equipos de mejor desempeño son mejores en las cuatro: velocidad y estabilidad no se excluyen.

La herramienta del curso, herramientas/entrega.py, calcula las tres primeras a partir del historial.

BLOQUE 1 · CULTURA

Lo que las herramientas no pueden hacer

Flujo

Hacer visible el trabajo y optimizar su paso de desarrollo a operación.

Retroalimentación

Detectar los problemas donde nacen, en las dos direcciones.

Aprendizaje continuo

Tratar cada incidente como información, con revisiones sin culpables.

Las adopciones de DevOps fracasan cuando se instalan pipelines sin cambiar los incentivos. Las tres vías son de Kim et al. (2016).

BLOQUE 1 · LA CADENA QUE SOSTIENE LA ENTREGA

Del repositorio a la evidencia

Repositorio
F-15 · convenciones y protección

›

Ramas
F-16 · estrategia justificada

›

Versiones
F-17 · SemVer y registro de cambios

›

Pipeline
F-18 · CI y CD

›

Evidencia
F-19 y F-20 · ejecuciones y métricas

El F-21 verifica antes de entregar. Cada decisión debe poder señalarse en el repositorio o en una ejecución del pipeline.

Ramas y versiones en Flask

Estrategia F-16	Rama de mantenimiento: **main** para la versión siguiente y **stable** para corregir la vigente. stable está 87 commits detrás de main y se integra a ella con frecuencia.
Ramas de trabajo F-16	Cortas, una por PR. En el remoto solo quedan otras dos ramas, de un commit cada una.
Versiones F-17	47 de 69 etiquetas cumplen versionamiento semántico. Las 22 que no: las antiguas de dos números (0.1 a 1.0), las que nombran ramas de mantenimiento (2.0.x) y **2.0.0rc1**, que en SemVer se escribe 2.0.0-rc1.
Contrato F-17	La 2.3.0, una versión menor, retiró código obsoleto. Bajo SemVer estricto eso sería una versión mayor: Flask avisa con un período de obsolescencia.

Las métricas de entrega de Flask desde 2021

Métrica	Valor	Lectura
Versiones por año	2021: 5 · 2022: 8 · 2023: 8 · 2024: 4 · 2025: 2 · 2026: 1	La frecuencia baja desde 2024, como mostró la Unidad 1
Tiempo de entrega (mediana)	61 días	Un cambio típico espera dos meses a la siguiente versión
Tiempo de entrega (percentil 75)	172 días	Una cuarta parte espera casi seis meses
Versiones de corrección	22 de 28 (79 %)	Aproximación gruesa: en una biblioteca, una corrección no siempre es un fallo
Commits con Conventional Commits	0 %	No es una convención del proyecto: el contrato está en CHANGES.rst

Cómo leerlo

Flask es una biblioteca: «desplegar» es publicar una versión.

Para un servicio, la etiqueta debería coincidir con el despliegue a producción.

Las métricas sirven para comparar el sistema consigo mismo en el tiempo.

Cada commit cuenta en la primera versión que lo publicó, de cualquier rama. Detalle en ejemplo_flask/entregas.csv.

La plantilla del curso

Qué trae

Un servicio mínimo en Flask con /salud y /version.

Pruebas, linter y formato.

Dockerfile con usuario sin privilegios.

CHANGELOG.md y scripts de verificación.

Dos flujos: CI y entrega.

Qué decide cada grupo

Reemplazar el servicio por su sistema.

La estrategia de ramas y la protección de main.

Quién aprueba el entorno de producción.

Qué pruebas hacen falta para que el pipeline signifique algo.

La plantilla resuelve la mecánica para que el grupo dedique las 24 horas a las decisiones. La calificación está en las decisiones y en la evidencia.

Carpeta plantilla/ del repositorio. Se copia a la raíz del repositorio del sistema.

El pipeline de CI, corriendo en GitHub

1. **calidad**
   ruff revisa errores comunes y formato. En la primera prueba local detectó tres líneas largas en un script de la propia plantilla.
2. **pruebas**
   pytest en Python 3.11, 3.12 y 3.13; los resultados quedan como artefactos de la ejecución, aunque fallen.
3. **imagen**
   Construye la imagen con el commit adentro, la levanta y comprueba que /version devuelve ese commit.

Evidencia real

Ejecución 41256889959 del repositorio del curso, el 1 de noviembre de 2026.

Los cinco trabajos en verde, en unos 40 segundos.

Esto es lo que la actividad pide mostrar.

plantilla/.github/workflows/ci.yml. Las acciones externas están fijadas por hash de commit, como recomendó la Unidad 2.

El pipeline de entrega: una etiqueta, cinco pasos

1. **verificar**
   La etiqueta tiene formato vMAYOR.MENOR.PARCHE, coincide con la versión declarada y el registro de cambios tiene su sección.
2. **pruebas**
   Se repiten sobre lo que se va a entregar.
3. **publicar-imagen**
   Una sola construcción, publicada en GitHub Container Registry con la versión y el commit como etiquetas.
4. **staging**
   La imagen publicada, identificada por su digest, se levanta y pasa la prueba de humo.
5. **produccion**
   Espera la aprobación configurada en el entorno y publica la entrega con las notas del registro de cambios.

plantilla/.github/workflows/entrega.yml. En la plantilla, producción es publicar la entrega; si el sistema tiene un servidor, ese paso despliega allí.

EL HACER · TALLER EN SALAS

De la sesión salen con el primer pipeline en verde

1	**Crear o preparar el repositorio del sistema** GitHub o GitLab, con la estructura y el .gitignore del F-15.	5 min
2	**Copiar la plantilla y adaptarla** Reemplazar el servicio o ajustar los comandos de prueba y construcción al lenguaje del sistema.	10 min
3	**Proteger main** Exigir PR y que la CI pase antes de integrar.	5 min
4	**Abrir el primer PR** Con un cambio pequeño, y ver correr el pipeline.	5 min
5	**Registrar la evidencia** Enlace a la ejecución y captura en el F-19.	5 min

Si el sistema no es Python, se conserva la estructura de los trabajos y se cambian los comandos.

PARA TENER AL LADO DURANTE EL TALLER

Comandos y configuración

Comandos

git switch -c feat/mi-cambio

git commit -m "feat(api): agrega la ruta /estado"

git tag -a v0.2.0 **-m** "Versión 0.2.0" y **git push --tags**

python scripts/verificar_version.py v0.2.0

python herramientas/entrega.py sistema

Configuración en GitHub

Settings → Branches: proteger main, exigir PR y la verificación CI.

Settings → Environments: crear staging y produccion; en produccion, quién aprueba.

Settings → Actions: permitir que los flujos publiquen paquetes.

CIERRE · QUÉ SIGUE

De automatizar la entrega a mejorar el sistema

ESTA SEMANA

Actividad 3

Repositorio, estrategia de ramas, versionamiento semántico, pipeline funcionando y evidencias. Verificar con el F-21 antes de entregar.

UNIDAD 4

Estrategias de mantenimiento y evolución

Con el pipeline como red de seguridad, se refactoriza y se moderniza el sistema, y se mide el impacto de cada mejora.

Un pipeline confiable es lo que hace posible refactorizar sin miedo. La Unidad 4 lo va a necesitar.

REFERENCIAS

Fuentes de la sesión

Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book

Conventional Commits. (s. f.). Conventional Commits 1.0.0. https://www.conventionalcommits.org

Forsgren, N., Humble, J., & Kim, G. (2018). Accelerate: The science of lean software and DevOps. IT Revolution Press.

GitLab. (2024). CI/CD documentation. https://docs.gitlab.com/ee/ci/

Humble, J., & Farley, D. (2010). Continuous delivery: Reliable software releases through build, test, and deployment automation. Addison-Wesley.

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps handbook. IT Revolution Press.

Preston-Werner, T. (s. f.). Semantic Versioning 2.0.0. https://semver.org

Pallets. (2026). Flask [Repositorio de código fuente]. https://github.com/pallets/flask